



Ministério da Educação
Universidade Tecnológica Federal do Paraná
Câmpus Santa Helena
Bacharelado em Ciência da Computação



TRABALHO DE PROJETO E DESENVOLVIMENTO DE SISTEMAS WEB

Portal Voluntário

Documentação

Bruno Eduardo Santos Alcantara
José Carlos Seben de Souza Leite
Fernando Buligon

Santa Helena
08/2026



SUMÁRIO

1. Introdução	2
2. Objetivos	3
3. Público e perfis de usuário	4
4. Funcionalidades	5
5. Telas e Navegação	6
6. Autenticação e controle de acesso	7
7. Arquitetura e organização do código	7
8. Tecnologias Utilizadas	8
9. Como executar o projeto	8



1. Introdução

O Portal Voluntário é um site que conecta a comunidade da UTFPR a oportunidades de voluntariado dentro e fora do campus. A proposta é reunir em um só lugar atividades ligadas aos três eixos da universidade (Ensino, Pesquisa e Extensão) e acompanhar a pessoa voluntária desde a descoberta de uma vaga até o registro das horas e dos certificados.

A ideia que guia o projeto aparece logo na página inicial: “Seu tempo. Um mundo de possibilidades.” Quem quer ajudar encontra, no portal, quem precisa de ajuda. Uma pessoa pode procurar uma atividade compatível com seus interesses e sua disponibilidade, se inscrever, ver os encontros na própria agenda e acompanhar, em um painel, tudo o que já realizou.

O catálogo de demonstração mostra o tipo de oportunidade que o portal reúne: uma horta comunitária com educação ambiental, uma monitoria de programação para iniciantes oferecida pelo Campus Santa Helena, um mapeamento da qualidade do Rio Paraná feito por um grupo de pesquisa, um clube de leitura infantil na biblioteca municipal e um projeto de dados abertos para impacto social. Cada oportunidade informa o eixo, a organização responsável, o formato (presencial, híbrido ou online), o local, as datas, a carga horária e quantas vagas já foram preenchidas.

O uso do portal segue uma jornada de três passos, apresentada na página inicial: primeiro a pessoa conta sua história, criando uma conta; depois encontra sua causa, explorando o catálogo e se inscrevendo; por fim, faz a diferença, participando dos encontros e acompanhando suas atividades, horas e certificados.

Este documento descreve a etapa de front-end do projeto. A interface foi construída pensando na próxima etapa da disciplina, o back-end, para que as mesmas telas possam ser conectadas a um servidor definitivo sem retrabalho.



2. Objetivos

Objetivo geral

Desenvolver uma plataforma web que conecte a comunidade da UTFPR a oportunidades de voluntariado em Ensino, Pesquisa e Extensão, facilitando tanto a participação nas atividades quanto o acompanhamento do que cada pessoa já realizou.

Objetivos específicos

- Centralizar, em um único catálogo público, as oportunidades de voluntariado oferecidas dentro e fora da universidade;
- Facilitar a escolha das atividades, apresentando de forma clara o eixo, a organização responsável, o formato, o local, as datas, a carga horária e as vagas disponíveis;
- Simplificar a inscrição e o cancelamento em atividades, liberando a vaga automaticamente para outra pessoa quando alguém desiste;
- Ajudar o voluntário a organizar sua rotina, reunindo em uma agenda os encontros de todas as atividades em que está inscrito;
- Auxiliar o estudante no controle das suas horas, mostrando em um painel o banco de horas e a distribuição das horas entre Ensino, Pesquisa e Extensão;
- Facilitar o registro e a guarda dos certificados de atividades externas, permitindo consultar e baixar os comprovantes sempre que necessário;
- Apoiar as organizações no controle de presença dos participantes e no envio dos certificados;
- Proteger os dados pessoais dos usuários, restringindo a área privada a quem está autenticado;
- Garantir que o portal possa ser usado por todos, com navegação pelo teclado e mensagens de erro acessíveis.



3. Público e perfis de usuário

O portal atende três perfis, sendo eles:

- O visitante é quem ainda não tem conta. Ele pode conhecer a proposta do portal, consultar todo o catálogo de oportunidades e abrir a demonstração da área de gestão. Quando tenta se inscrever em uma atividade, é levado à tela de login e, depois de entrar, volta automaticamente para a página onde estava.
- O voluntário é quem criou uma conta pelo cadastro público. Além de tudo o que o visitante faz, ele pode se inscrever e cancelar inscrições, acompanhar os encontros em uma agenda, registrar certificados de atividades externas e acompanhar suas horas no painel.
- A organização é o perfil das instituições que oferecem as atividades. Ela tem acesso à área de gestão, onde acompanha a presença dos participantes e envia os certificados. Esse perfil não pode ser obtido pelo cadastro público: ele deverá ser concedido por um processo administrativo quando o back-end estiver disponível. Para que todos possam conhecer essa área, o portal oferece uma demonstração pública com dados fictícios.



4. Funcionalidades

Catálogo de oportunidades: É a vitrine do portal. Lista as atividades de Ensino, Pesquisa e Extensão com as informações necessárias para a pessoa decidir se quer participar. O catálogo é público, mas a inscrição exige login.

Cadastro e login: O cadastro pede nome, e-mail e senha, com confirmação. Todo cadastro público cria uma conta de voluntário. O login dá acesso à área privada e mantém a pessoa conectada por até oito horas.

Inscrições e agenda: O voluntário pode se inscrever em uma atividade e cancelar a inscrição; o cancelamento pede confirmação e libera a vaga para outra pessoa. Os encontros das atividades inscritas aparecem em um calendário, na página “Minhas atividades”.

Painel: Reúne a situação do voluntário em uma só tela: o banco de horas, as horas por eixo, as inscrições ativas, os certificados lançados e as próximas atividades.

Certificados. O voluntário pode registrar atividades externas, informando os dados da atividade e anexando o comprovante em PDF, JPG ou PNG de até 5 MB. Os registros podem ser consultados e baixados depois. Nesta etapa, o recurso funciona como demonstração: os registros ficam salvos no próprio navegador e permanecem “em análise”, sem somar horas homologadas.

Gestão de atividades: Protótipo da área das organizações, onde é possível marcar a presença dos participantes e simular o envio de certificados. A versão pública de demonstração usa participantes fictícios e reinicia ao recarregar a página.

Acessibilidade: O portal pode ser usado pelo teclado, oferece um atalho “Ir para o conteúdo”, associa as mensagens de erro aos campos dos formulários e reduz as animações para quem prefere menos movimento na tela.



5. Telas e Navegação

O portal tem nove telas: cinco públicas e quatro privadas. A página inicial tem um cabeçalho com os links Início, Vagas, Demonstração e Entrar. As demais telas usam um menu lateral, que se adapta ao perfil de quem está navegando: o visitante vê a opção de entrar, o voluntário vê suas áreas, e a organização vê também a gestão de atividades.

Rota	Acesso	Descrição
/	Público	Página inicial, com a proposta do portal e atalhos para o catálogo
/oportunidades	Público	Catálogo de oportunidades; a inscrição exige login
/demonstracao/gestao	Público	Demonstração da área de gestão, com dados fictícios
/entrar	Público	Login
/criar-conta	Público	Cadastro de voluntário
/painel	Privado	Banco de horas, inscrições, certificados e próximas atividades
/minhas-atividades	Privado	Agenda e cancelamento de inscrições
/certificados	Privado	Registro, histórico e download de certificados
/organizacao	Privado (organização)	Presença dos participantes e envio de certificados



6. Autenticação e controle de acesso

O login acontece na tela “Entrar”. O formulário valida o e-mail e a senha assim que o usuário sai de cada campo e mostra as mensagens de erro junto do campo correspondente. Ao enviar, o servidor valida os dados de novo e, se estiverem corretos, cria uma sessão e grava um cookie no navegador. Esse cookie é do tipo HttpOnly, ou seja, não pode ser lido por scripts da página, o que dificulta o roubo da sessão.

A proteção das rotas privadas fica no arquivo `src/proxy.ts`, que na versão 16 do Next.js substitui o antigo middleware. Ele é executado antes de qualquer página privada ser carregada e verifica se existe uma sessão válida. Se não existir, o usuário é redirecionado para a tela de login, e o endereço que ele tentou acessar fica guardado; depois do login, ele volta exatamente para esse endereço. Se a pessoa estiver logada, mas tentar abrir a área de gestão sem ter o perfil de organização, é redirecionada para o painel.

Como camada extra de segurança, cada página privada confere a sessão novamente no servidor antes de exibir os dados. Assim, mesmo que uma verificação falhe, a outra impede o acesso indevido.

Outras medidas reforçam a autenticação: as senhas são armazenadas com o algoritmo `bcrypt`, nunca em texto puro; sair da conta encerra a sessão também no servidor; tentativas de login repetidas para o mesmo e-mail são bloqueadas temporariamente; e o servidor só aceita alterações enviadas pelo próprio portal.



7. Arquitetura e organização do código

O código segue o padrão Feature-Based, em que cada área do sistema fica em sua própria pasta dentro de `src/features`. Há pastas para autenticação, oportunidades, painel, certificados, organizações e página inicial. Os componentes usados por várias áreas, como os layouts e os componentes visuais básicos, ficam em `src/components`; as rotas da aplicação ficam em `src/app`.

Dentro de cada área, as responsabilidades são separadas em camadas. Os componentes cuidam apenas da interface e de exibir os estados da tela: carregando, vazio, erro e sucesso. Os hooks guardam o estado da tela e as ações do usuário. Os serviços fazem as chamadas à API e tratam os erros. Por fim, os contratos, escritos com a biblioteca Zod, descrevem o formato dos dados e validam tudo o que chega de fora antes que chegue à interface.

Essa separação aplica diretamente os conceitos da disciplina. A interface é dividida em componentes pequenos e reutilizáveis, como os botões, os cartões e os campos de formulário. O estado local fica nos hooks de cada tela, criados com `useState` e `useEffect`, e os formulários usam a biblioteca `React Hook Form`. A navegação usa o `App Router` do `Next.js`, em que cada pasta corresponde a uma rota.

A mesma separação é o que prepara o projeto para o back-end. Como as telas não chamam a API diretamente, a fonte dos dados pode mudar sem que a interface precise ser refeita. Hoje, o login, o cadastro e as inscrições já usam endpoints reais do próprio `Next.js`, enquanto os recursos que ainda não têm servidor, como certificados e gestão, são simulados no navegador com a biblioteca `MSW`. Na próxima etapa, basta implementar esses endpoints no back-end e desligar a simulação.



8. Tecnologias Utilizadas

A aplicação foi construída com Next.js 16 e React 19, usando TypeScript em modo estrito em todo o código. A interface usa Tailwind CSS e componentes do shadcn/ui, seguindo a identidade visual da UTFPR, com preto e amarelo como cores principais. Os formulários usam React Hook Form com validação feita pela biblioteca Zod. As animações são feitas com Motion, e os ícones vêm do pacote lucide-react. A simulação da API usa MSW. As dependências são gerenciadas com pnpm, e a integração contínua roda no GitHub Actions.



9. Como executar o projeto

É preciso ter o Node.js 22.12 ou superior e o pnpm instalados. O pnpm pode ser ativado pelo Corepack, que já vem com o Node.js:

```
None  
corepack enable pnpm
```

Antes de iniciar, crie um arquivo chamado `.env.local` na raiz do projeto com uma chave secreta usada para assinar as sessões. A chave deve ter pelo menos 32 caracteres e pode ser gerada com o comando abaixo:

```
None  
node -e  
"console.log(require('crypto').randomBytes(32).toString('base64'))"
```

O conteúdo do arquivo deve ser uma única linha, no formato `SESSION_SECRET=chave-gerada`.

Depois, instale as dependências e inicie o servidor de desenvolvimento:

```
None  
pnpm install  
pnpm dev
```

O portal fica disponível em <http://localhost:3000>. Para acessar a área privada, crie uma conta na tela de cadastro e entre com ela.

O projeto também oferece comandos de verificação: `pnpm lint` analisa o código, `pnpm typecheck` confere os tipos, `pnpm test` executa os testes automatizados e `pnpm build` gera a versão de produção. O comando `pnpm check` executa todos eles em sequência.